Project 1 实验报告

CS30064.01 Neural Network and Deep Learning

姓名：洪家权

学号：23307110248

1. 引言

1.1.实验架构简述

本报告围绕 MNIST 手写数字分类任务展开。选择的实验架构如下：

- **Part A**：numpy 实现全连接算子并构建基线 MLP；
- **Part B**：numpy 实现卷积算子并构建基线 CNN，与 MLP 进行对比；
- **Part C**：
  - **Optimization**：
    - 实现optimizer.py中的动量(**MomentGD**)组件、lr_scheduler.py中的阶梯衰减学习率(**StepLR**)组件与指数衰减学习率(**ExponentialLR**)组件；
    - 将三个组件分别单独与MLP/CNN的基线模型组合(**基线+Momentum/基线+StepLR/基线+ExponentialLR**)，与基线模型对比训练性能；
  - **Regularization**：
    - 在原版的runner.py的基础上添加早停(**Early stopping**)设计,在models.py中加入L2正则化选项(相应地在optimizer.py等处添加了L2正则化项的更新逻辑)；
    - 考察MLP/CNN的三种正则化设置组合(**基线+早停/baseline+L2正则化/基线+早停+L2正则化**)的训练情况，与基线模型对比训练性能；
  - **Robustness Analysis**：对测试集图片添加不同强度的高斯噪声($\epsilon \sim \mathcal{N}(0, \sigma^2)$**,** $\sigma \in \{0,\ \ 0.01, 0.05, 0.1\}$),评估MLP/CNN的基线模型，检验模型稳定性的同时探索模型的抗干扰能力差异；
  - **Error Analysis and Visualization**:绘制MLP/CNN基线模型的混淆矩阵，并选取典型错分样例，深入分析两类网络的错误模式。

1.2.实验数据集说明

实验数据集使用 PJ1 文件提供的 MNIST 手写体数据集。测试集为给定的 10000 张图像；训练集共 60000 张图像，其中 50000 张用于训练，10000 张用于验证。值得注意的是，为保证不同实验组之间的可比性，报告中涉及到的所有模型训练行为统一使用同一个 **idx.pickle** 划分索引文件，从而确保每次训练的训练集/验证集划分一致。

1.3.实验超参数设置

为保证MLP与CNN公平比较，一方面，所有模型统一采用以下设置：

- **(initial) learning rate**：0.01；

- **batch size**：128；
- **epochs**：40；
- 每 50 个 mini-batch step 在验证集与测试集上评估一次。

另一方面，实验控制MLP与CNN网络模型的**可训练参数量**一致。现规定网络结构的**书写格式**：

- 卷积层：Conv(输入通道数->输出通道数，卷积核尺寸)（**步长默认1**）；
- 全连接层：FC(输入维数->输出维度数)；
- 最大池化：MaxPool(池化核尺寸,步长)。

下面给出MLP与CNN结构设计细节与可训练参数总量计算过程。

- MLP：FC(784 -> 260) + ReLU + FC(260 -> 10) + Softmax
  - input-hidden: 784 * 260 + 260 = 204100
  - hidden-output: 260 * 10 + 10 = 2610
  - total: 206,710
- CNN：Conv(1 -> 16,3 * 3) + ReLU + MaxPool(2 * 2,2) + Conv(16 -> 32,3 * 3) + ReLU + MaxPool(2 * 2,2) + flatten + FC(32 * 7 * 7 -> 128) + ReLU + FC(128 -> 10)
  - conv1: 16 * (1 * 3 * 3) + 16 = 160
  - conv2: 32 * (16 * 3 * 3) + 32 = 4,640
  - fc1: 1568 * 128 + 128 = 200,832
  - fc2: 128 * 10 + 10 = 1,290
  - total: 206,922

可以看到，MLP与CNN的可训练总参数量差异仅仅占总参数量的约0.1%。可以认为，两者在可训练参数规模这一维度上是公平的。

2. Part A & B: MLP & CNN baseline

本节聚焦基线模型组件的数学实现解释，并结合训练/验证/测试曲线分析 MLP 与 CNN 基线模型的性能，解释 CNN 在 MNIST 手写体识别任务上性能优于 MLP 的原因。鉴于第 1 节已经具体给出数据划分、超参数与可训练参数量公平性设置，下文不再赘述。

2.1 Softmax 与交叉熵

MLP 与 CNN 都是多分类模型，最终输出均为 logits 类别分数 $z \in \mathbb{R}^{B\times C}$ (尚未经softmax转为概率)。其中B对应batch_size，C对应类别数。这些输出经由op.py中的 `MultiCrossEntropyLoss` 完成 softmax 与交叉熵计算。

对第 i 个样本、第 c 类，softmax 定义为

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{k=1}^{C} \exp(z_{i,k})}$$

其中 $p_{i,c}$ 为第 i 个样本被预测为类别 c 的概率。在具体实现中，代码使用了数值稳定写法（先减去每行最大值）避免指数上溢。

对真实标签 y_i，批量平均交叉熵为

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\log p_{i,y_i}$$

其中 p_{i,y_i} 为第 i 个样本在真实标签上的预测概率。将 one-hot 标签记为 $\mathbf{y}_i$，则对 logits 的梯度可化简为

$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{1}{B}(\mathbf{p}_i - \mathbf{y}_i)$$

2.2 线性层与卷积层

为清晰表示层间关系，统一记第 i 层输入为 X_i，线性/卷积变换输出为 Y_i，非线性函数激活后输出为 $X_{i+1} = f(Y_i)$。

2.2.1 线性层

设第 i 个线性层参数为 W_i, b_i，且

$$X_i \in \mathbb{R}^{B\times d_i}, \quad W_i \in \mathbb{R}^{d_i\times d_{i+1}}, \quad b_i \in \mathbb{R}^{1\times d_{i+1}}$$

前向传播为

$$Y_i = X_i W_i + b_i, \qquad X_{i+1} = f(Y_i)$$

反向传播时，给定上游梯度 $\frac{\partial \mathcal{L}}{\partial X_{i+1}}$，先经激活函数链式法则得到

$$\frac{\partial \mathcal{L}}{\partial Y_i} = \frac{\partial \mathcal{L}}{\partial X_{i+1}} \odot f'(Y_i)$$

记 $G_i = \frac{\partial \mathcal{L}}{\partial Y_i}$，则

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial W_i} &= X_i^T G_i, \\
\frac{\partial \mathcal{L}}{\partial b_i} &= \sum_{n=1}^{B} G_i^{(n,:)}, \\
\frac{\partial \mathcal{L}}{\partial X_i} &= G_i W_i^T
\end{aligned}$$

将 $\frac{\partial \mathcal{L}}{\partial X_i}$ 的取值传给上一层，从而实现了链式地反向提供梯度信息。

2.2.2 卷积层

设第 i 个卷积层输入为 $X_i \in \mathbb{R}^{N\times C_{in}\times H\times W}$，卷积核为 $K_i \in \mathbb{R}^{C_{out}\times C_{in}\times k\times k}$，偏置为 $b_i \in \mathbb{R}^{C_{out}}$，步长为 s，填充为 p。前向传播写为

$$Y_i = \mathrm{Conv}(X_i; K_i, b_i, s, p), \qquad X_{i+1} = f(Y_i)$$

其逐元素形式为

$$Y_i[n, o, h, w] = \sum_{c,u,v} K_i[o, c, u, v]\, X_i^{pad}[n, c, hs + u, ws + v] + b_i[o]$$

反向传播同样先处理激活函数链式关系：

$$\Delta_i = \frac{\partial \mathcal{L}}{\partial Y_i} = \frac{\partial \mathcal{L}}{\partial X_{i+1}} \odot f'(Y_i)$$

随后得到卷积层参数与输入的梯度：

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial K_i} &= X_i \star \Delta_i, \\
\frac{\partial \mathcal{L}}{\partial b_i} &= \sum_{n,h,w} \Delta_i[n, :, h, w], \\
\frac{\partial \mathcal{L}}{\partial X_i} &= \Delta_i * \widetilde{K_i}
\end{aligned}$$

其中 $\star$ 表示与卷积核梯度对应的相关运算，$\widetilde{K_i}$ 表示空间翻转并交换输入/输出通道后的核。

在具体实现时，考虑到四重循环的效率过低，卷积层并未使用逐位置多重循环，而是采用 **im2col -> 矩阵乘法 -> col2im** 的向量化流程：前向将滑窗展开到列空间后与卷积核矩阵相乘；反向先在列空间计算 `dW` 与 `dX_cols`，再通过 `np.add.at` 将重叠窗口梯度正确累加回输入张量。以下python代码实现节选自 `op.py` 的 `conv2D`。

(变量含义：

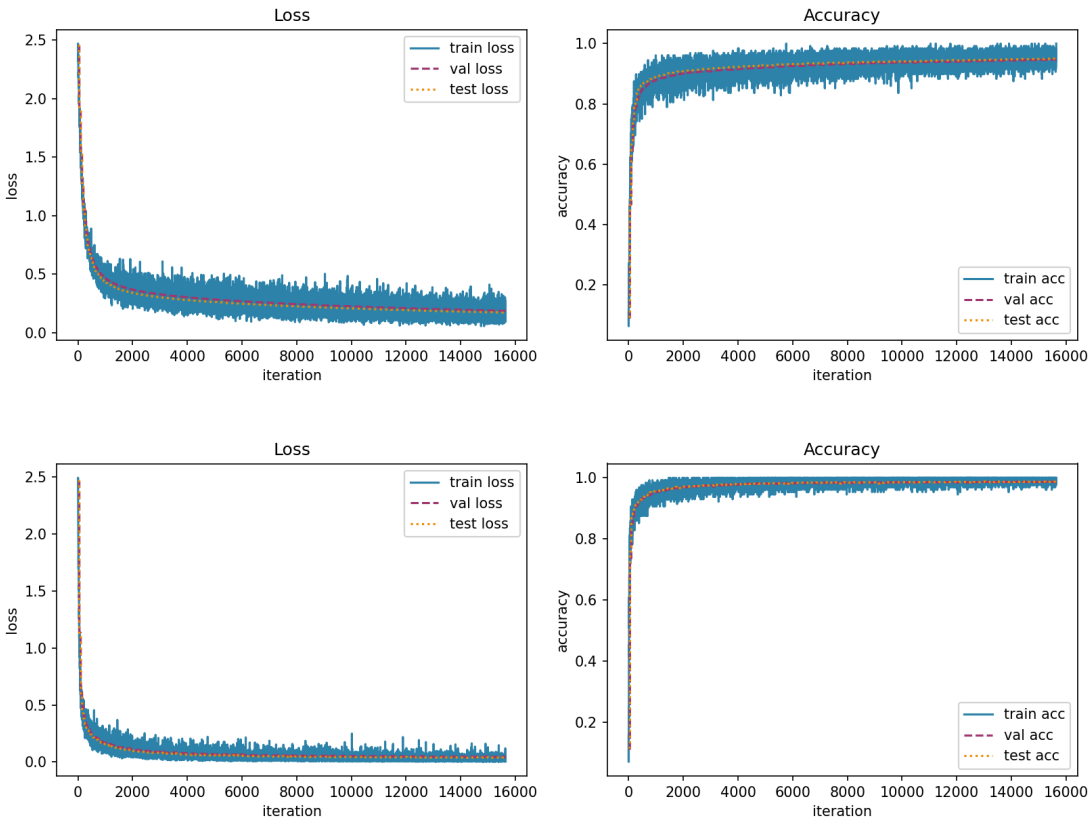
- `N` 为 batch size，`C` / `C_in` 为输入通道数，`C_out` 为输出通道数，`k` 为卷积核边长，`H_out,W_out` 为输出特征图尺寸；
- `X_pad` 为填充后的输入，`cols` 为 `im2col` 展开结果，`W_col` 为卷积核二维重排矩阵，`grad_col` 为输出梯度在列空间的重排结果，`dX_pad` 为填充输入对应梯度；
- `col_k,col_i,col_j`（或 `c_idx,i_idx,j_idx`）为 `im2col/col2im` 的索引映射。）

In [ ]:
```
# forward: im2col + GEMM
cols = X_pad[:, col_k, col_i, col_j] # [N, C*k*k, H_out*W_out]
cols = cols.transpose(0, 2, 1).\
    reshape(N * H_out * W_out, -1) # [N*H_out*W_out, C*k*k]
W_col = W.reshape(C_out, -1)
out = cols @ W_col.T + b

# backward: gradients in column space
grad_col = grad.transpose(0, 2, 3, 1).\
    reshape(N * H_out * W_out, C_out)
dW_col = grad_col.T @ cols
self.grads['W'] = dW_col.reshape(W.shape)
self.grads['b'] = np.sum(grad_col, axis=0, keepdims=True)
```

```
dX_cols = grad_col @ W_col

# col2im: accumulate overlaps back to input space
np.add.at(dX_pad,
        (np.arange(N)[:, None, None], c_idx[None, :, :],
        i_idx[None, :, :], j_idx[None, :, :]),
        dX_cols.reshape(N, H_out * W_out, -1).\
            transpose(0, 2, 1))
```

2.3 曲线解读与性能对比

从上到下依次为 MLP 与 CNN 在训练集、验证集与测试集上的 loss 与 accuracy 曲线。注意到，如第一节所述，出于效率考虑，每50轮迭代才在验证集与测试集上评估一次模型，因此 loss 与 accuracy 曲线不连贯。



在曲线形态上，MLP 与 CNN 都表现出相似的总体收敛规律：训练初期 loss 快速下降、accuracy 快速上升，随后逐步进入平台期；验证曲线与测试曲线在两类模型中都长期贴合，说明训练过程整体稳定、未出现明显异常分叉。差异在于，CNN 的 loss 曲线整体更低且下降更快，accuracy 更早达到高位并维持在更高平台；相比之下，MLP 在后期仍存在更明显的训练波动，表现为高精度区间内的抖动更强。可以初步判断 MLP 在 MNIST 手写体识别任务上，稳定性与准确性性能不及CNN。

训练记录(curves.npz)中的数据进一步印证了曲线解读中的判断：MLP baseline 的验证集准确率最高约为 **0.9468**，测试集准确率最高约为 **0.9502**，且在验证最优点对应的测试准确率为 **0.9498**；CNN baseline 的验证集准确率最高约为 **0.9876**，测试集准确率最高约为 **0.9879**，且在验证最优点对应的测试准确率为 **0.9871**。在统一训练协议与接近参数量条件下，CNN 相比 MLP 的验证峰值、测试峰值与验证峰值对应的测试准确率均提升约 **4%**。

2.4 CNN 优于 MLP 原因分析

CNN 在本任务中优于 MLP 的关键，并不是在于可训练参数更多(两者的可训练参数个数差异约为总数的千分之一，详见第一部分)，而是在于其归纳偏置与手写数字图像的统计结构高度一致。

对于 MNIST 这类 28×28 灰度图像，类别信息主要由局部笔画（横、竖、弯钩、交叉等）及其空间组合关系构成，卷积层通过局部感受野在小区域内建模，能够直接捕捉这些稳定的低层模式；同时，参数共享使同一个卷积核在整张图上复用，相当于将局部特征检测器应用到不同位置，从而显著降低自由参数、减少对数据规模的依赖，并提升特征的可迁移性；进一步地，网络由浅到深的层级结构会把边缘/角点上的局部信息，逐步组合为网络中的局部组件，局部组件再进一步组合为数字整体形状，这种逐层抽象通常比 MLP 在展平向量上直接学习全局映射更高效。

此外，在卷积操作的基础上，池化操作则在空间维度做压缩与不变性增强，使模型对轻微平移、笔画粗细变化、局部噪声扰动不敏感，降低了测试时输入微小偏移带来的性能波动，进一步提升了 CNN 在训练任务中的鲁棒性。相比之下，MLP 将图像展平后会弱化二维邻域关系，需要依赖全连接层来重新学习空间结构，容易把位置相关噪声一并记忆，导致达到同等泛化性能所需样本效率更低。

3. Part C：Additional Directions

在完成 Part A（MLP baseline）与 Part B（CNN baseline）后，本文进一步选择了四个补充方向：Optimization、Regularization、Robustness Analysis、Error Analysis and Visualization。选择这四个方向的原因如下：其一，Optimization 与 Regularization 分别对应**如何更快更稳地优化参数**与**如何控制泛化能力**两个训练核心问题，可用于检验基线模型在训练策略层面的可提升空间；其二，Robustness Analysis 用于评估模型在噪声扰动下的稳定性，能够补充关于最终得到的**测试集高精度是否可靠**的证据；其三，Error Analysis and Visualization 通过混淆矩阵与典型错分样例揭示模型的具体失效模式，帮助将定量结果转化为可解释结论。四个方向共同构成了从优化过程、泛化约束、抗扰动能力到误差模式的完整分析链路。

纵观整个Part C，如果要判断哪一个额外研究方向最有信息量，从性能改进与训练机制辨析角度看，最有信息量的是 **Optimization** 中在 baseline基础上引入 **momentGD** ，因为该设置在 MLP 与 CNN 上都带来一致增益，并与两种学习率衰减策略形成清晰对照；从可解释性角度看，最有信息量的是 **Error Analysis and Visualization**，因为其直接揭示了模型的高频混淆类别与典型难例类型，补足了仅凭 accuracy 难以呈现的模型行为信息。

3.1 Direction 1：Optimization

本节比较四种优化设置：`baseline`、`baseline + momentum`、`baseline + StepLR`、`baseline + ExponentialLR`。展示部分使用 CNN 的四组曲线图（ MLP 的曲线图见**codes/figs/mlp/Optimization**文件夹）。

其中，Momentum 的基本思想是在参数更新中引入速度项，对历史梯度做指数加权平均：

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

这样可在方向较稳定时加速收敛，并抑制小批量梯度噪声导致的震荡。

StepLR 采用分段常数式衰减，在固定步长间隔将学习率按比例缩小：

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

其中 s 为衰减步长。

ExponentialLR 则在每次参数更新后连续指数衰减学习率：

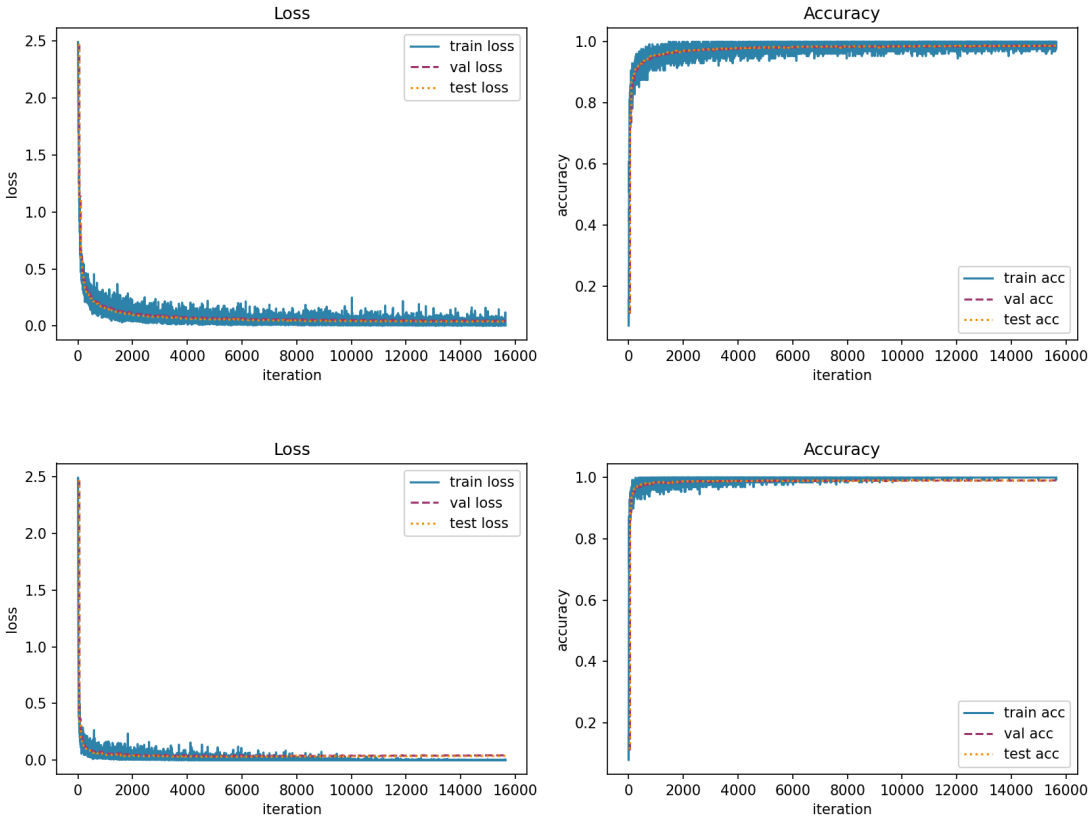
$$\eta_t = \eta_0 \cdot \gamma^t$$

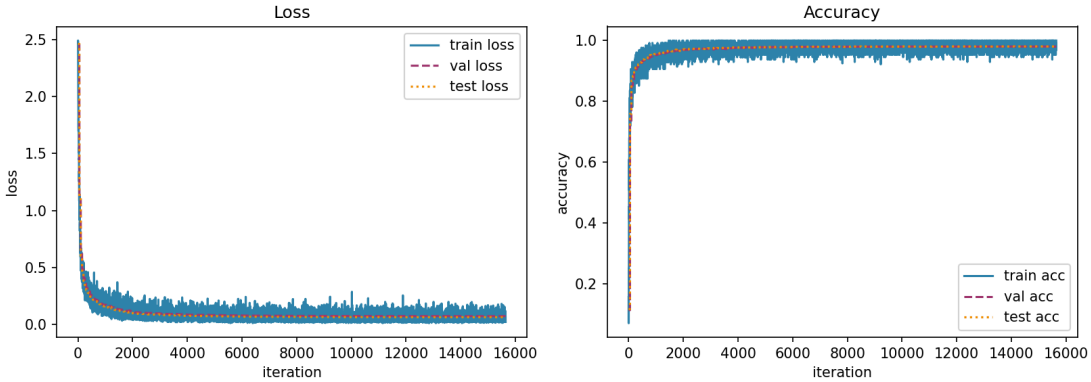
其衰减更平滑但也更敏感：若 γ 过小，会导致学习率过早降到很低水平。

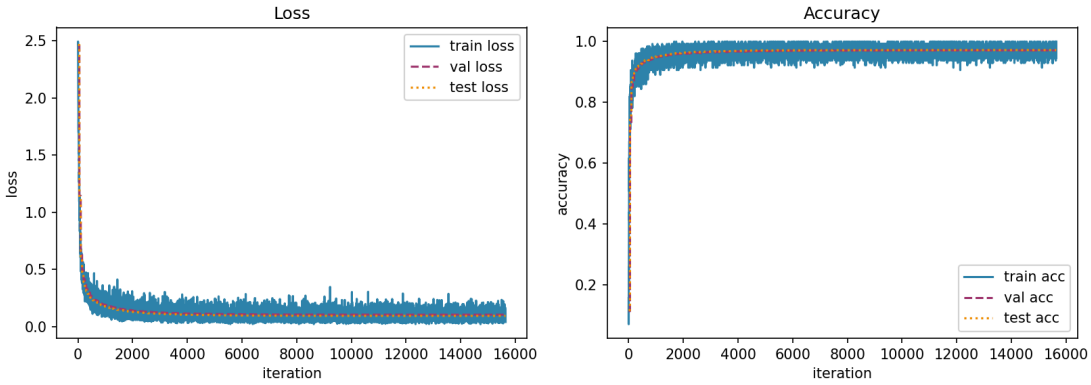
超参数设置如下：

- momentum：$\mu = 0.9$；
- StepLR：$s = 2000, \gamma = 0.5$；
- ExponentialLR：$\gamma = 0.9995$。

下列四张图，从上到下分别为baseline、baseline + momentum、baseline + StepLR、baseline + ExponentialLR 的 loss 与 accuracy 曲线。







从 CNN 四组曲线可见，`momentum` 组在前期收敛速度和后期 accuracy 平台上都优于 baseline，loss 平台也更低；`StepLR` 与 `ExponentialLR` 两组在当前超参数下下降更早放缓，最终 accuracy 平台明显低于 baseline。换言之，本实验设置下 `momentum` 带来增益最明显，而两种学习率衰减策略并未带来正向提升，反而出现了不同程度的欠拟合迹象。

下表为查阅各次实验的curves.npz后得到的数据，其中包括验证集准确率峰值（`dev_max`）、测试集准确率峰值（`test_max`）与验证峰处测试集准确率（`test@best_dev`）。

模型	设置	dev_max	test_max	test@best_dev
MLP	baseline	0.9468	0.9502	0.9498
MLP	momentum	0.9776	0.9804	0.9788
MLP	StepLR	0.9161	0.9241	0.9240
MLP	ExponentialLR	0.9029	0.9078	0.9078
CNN	baseline	0.9876	0.9879	0.9871
CNN	momentum	0.9913	0.9916	0.9890
CNN	StepLR	0.9806	0.9809	0.9804
CNN	ExponentialLR	0.9711	0.9723	0.9716

综合图表与数值可以相互印证，在当前实验配置下，采用 MomentumGD 对 MLP 与 CNN 均显著有效；引入 StepLR 与 ExponentialLR 策略的效果则不及 baseline 的恒定学习率策略。这可能主要是因为从实验设置的初始学习率出发($\eta = 0.01$),当前衰减强度与调用频率组合后，学习率在训练中前期就下降过快，参数更新步长迅速变小。而模型还未充

分逼近更优区域时，学习率已经衰减到接近 0，导致 loss 提前停在较高平台、accuracy 上升受限。该现象在 MLP 与 CNN 上都表现为训练集与验证集准确率同时偏低，更符合欠拟合而非过拟合的特征。

3.2 Direction 2：Regularization

本节比较四种设置：`baseline`、`baseline + early stopping`、`baseline + L2 regularization`、`baseline + L2 regularization + early stopping`。展示部分使用 CNN 的四组曲线图（MLP 的曲线图见**codes/figs/mlp/Regularization**文件夹）。

正则化设置广义上可以被解释为，在经验风险最小化的基础上，加入结构惩罚项或停止准则。

L2 正则化对应目标函数

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \|\theta\|_2^2$$

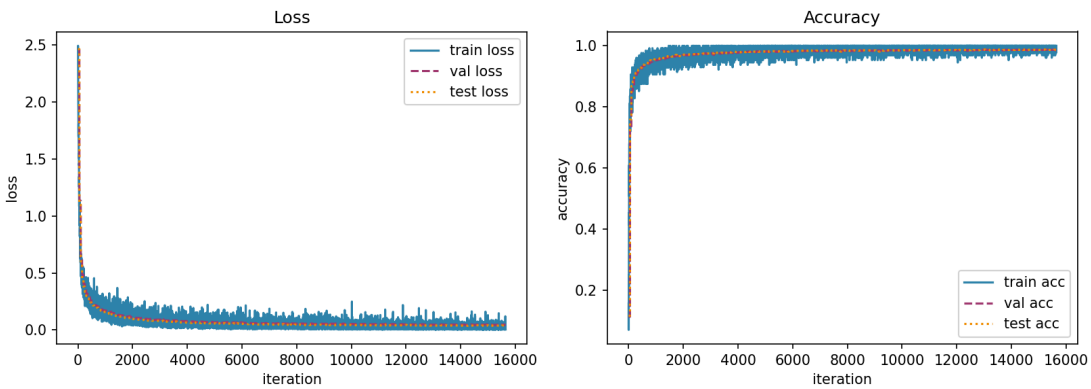
其中 $\lambda > 0$ 控制参数惩罚强度；

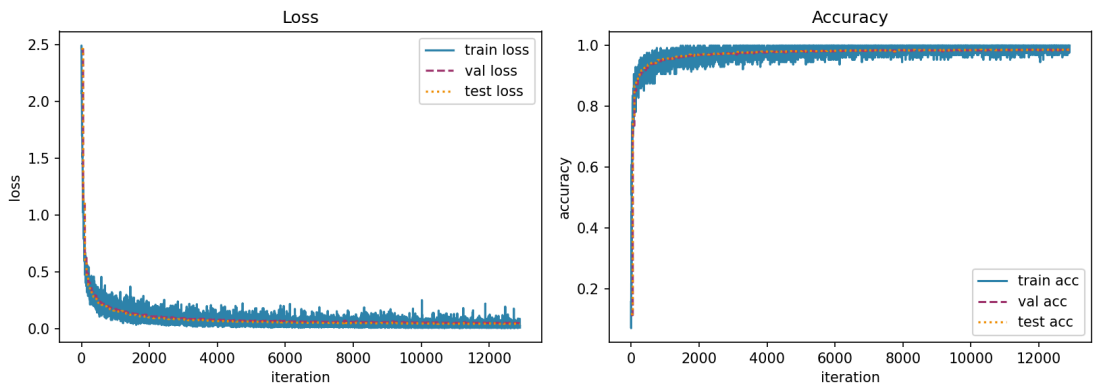
Early Stopping 则可表示为：若验证集指标在连续 p 个 epoch 内无提升，则提前终止训练，以限制后期无效拟合。

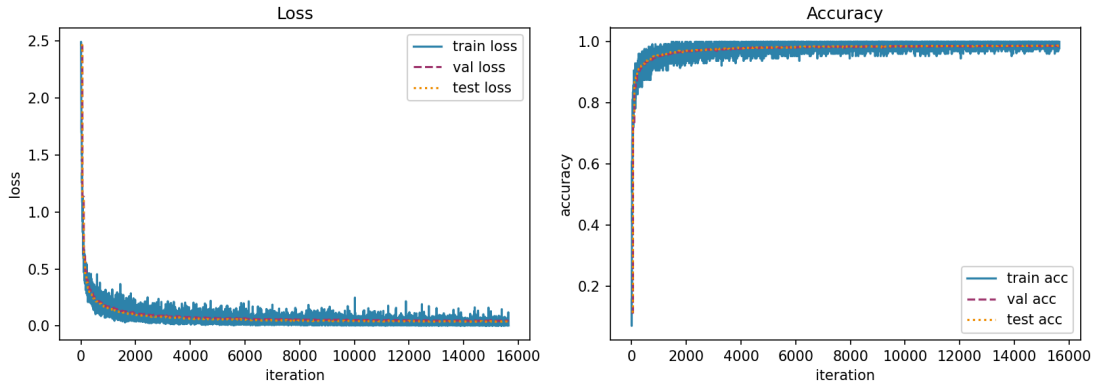
超参数设置如下：

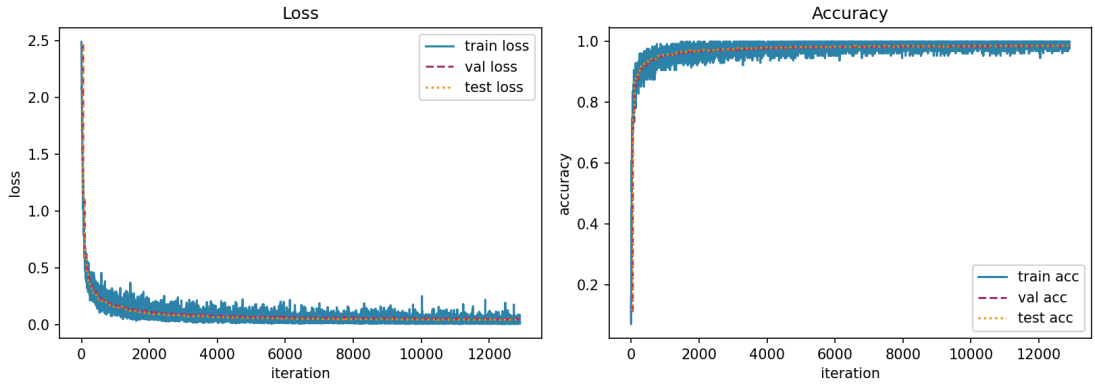
- early stopping：$p = 5$；
- L2 regularization：$\lambda = 1 \times 10^{-5}$。

下列四张图，从上到下分别为 baseline、baseline + early stopping、baseline + l2 regularization、baseline + l2 regularization + early stopping 的 loss 与 accuracy 曲线。









从 CNN 四组曲线看，L2 正则化组前期的 loss 与accuracy 与基线接近，后期测试曲线略有提升；早停组与 L2 正则化 + 早停组在当前设置下没有明显优于 baseline，验证与测试平台都略低于 baseline。整体上，本组正则化策略对 MLP 和 CNN 的提升幅度都小于 Optimization 中 momentum 带来的提升，说明在当前超参数设置下，以本实验的模型容量与数据规模，基线模型已较稳定，额外正则化在提升准确性这一维度上能够得到的收益较为有限。

下表为查阅各次实验的curves.npz后得到的数据，其中包括验证集准确率峰值（`dev_max`）、测试集准确率峰值（`test_max`）与验证峰处测试集准确率（`test@best_dev`）。

模型	设置	dev_max	test_max	test@best_dev
MLP	baseline	0.9468	0.9502	0.9498
MLP	earlyStop	0.9468	0.9502	0.9498
MLP	L2	0.9468	0.9501	0.9497

模型	设置	dev_max	test_max	test@best_dev
MLP	L2+earlyStop	0.9468	0.9501	0.9497
CNN	baseline	0.9876	0.9879	0.9871
CNN	earlyStop	0.9867	0.9872	0.9860
CNN	L2	0.9876	0.9882	0.9872
CNN	L2+earlyStop	0.9870	0.9870	0.9865

综合表格数据可以发现，单独 L2 正则化在 CNN 上带来轻微正向改进，但对 MLP 基本无影响；早停和早停 + L2 正则化操作在当前超参数设置下的准确性性能未超过基线。这一结果符合对于 L2 正则化与早停操作的预期。

早停的主要优势，主要不是在于提升最终网络模型的准确率，而是更多体现在训练过程中及早识别 accuracy 平台，提前终止训练从而达成的、对训练性价比的提升。根据各组 `curves.npz` 的训练步数换算（每个 epoch 约 391 个 mini-batch），四组实验的实际训练 epoch 如下：

设置	MLP 训练 epoch	CNN 训练 epoch
baseline	40	40
early stopping	40	33
L2 regularization	40	40
L2 + early stopping	40	33

可以看到，在当前超参数配置下，对 MLP 而言，验证指标在训练后期仍有缓慢改进，早停条件未触发，因此效率收益不明显；但对 CNN 而言，引入早停设置将 CNN 的训练轮次从 40 降到 33（约减少 17.5% 训练开销），而准确率仅比未早停组略低，体现了早停操作运用更少训练成本，达到接近性能上限的重要价值。

3.3 Direction 4：Robustness Analysis

为验证基线MLP和CNN模型的鲁棒性，本节采用高斯噪声扰动测试集图片：$\epsilon \sim \mathcal{N}(0, \sigma^2)$，$\sigma \in \{0, 0.01, 0.05, 0.1\}$。四种$\sigma$取值对应无噪声对照组、轻微噪声、中等噪声与较强噪声。为保证可复现性，实验使用固定 10 个随机种子（309 - 318）重复评估，并统计均值与标准差。

下表基于 `RobustnessAnalysis.json` 汇总得到：

σ	MLP acc (mean ± std)	CNN acc (mean ± std)	MLP drop(mean)	CNN drop(mean)
0.00	0.9498 ± 0.0000	0.9871 ± 0.0000	0.0000	0.0000
0.01	0.9498 ± 0.0002	0.9868 ± 0.0001	0.0000	0.0003
0.05	0.9491 ± 0.0007	0.9861 ± 0.0006	0.0007	0.0010
0.10	0.9462 ± 0.0006	0.9834 ± 0.0007	0.0036	0.0036

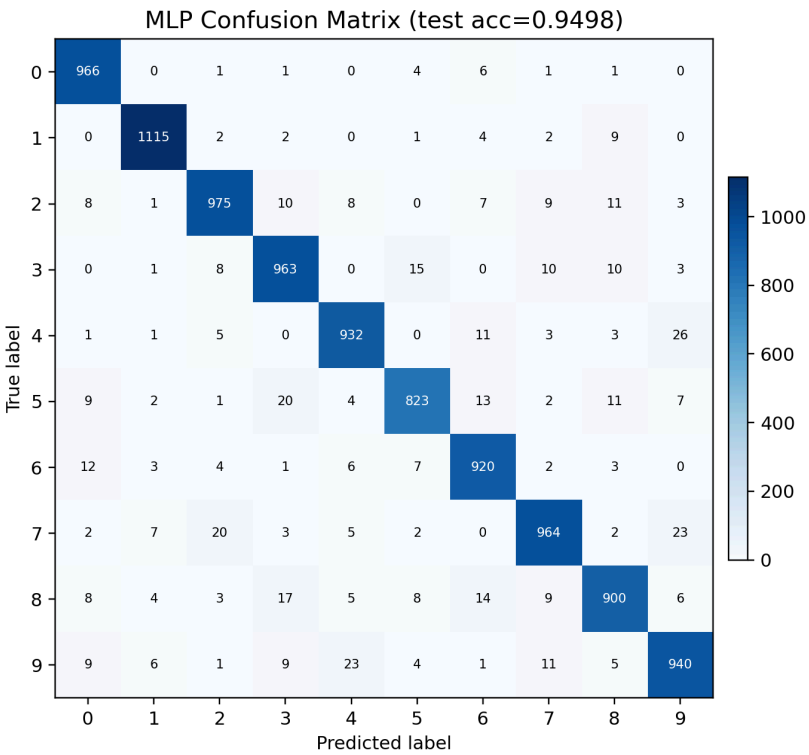
观察上表可知，一方面，随着噪声强度 σ 增大，MLP 与 CNN 的准确率均呈下降趋势，说明高斯扰动会持续削弱模型判别能力，且在较大噪声下影响更明显；另一方面，在相同噪声水平下，CNN 的绝对准确率仍旧始终高于 MLP，但从相对下降量（drop）看，CNN 在中等噪声影响下（$\sigma = 0.05$）的下降幅度略大于 MLP，而在高噪声影响下（$\sigma = 0.1$）两者下降幅度接近，这意味着，CNN 在本实验中的相对抗噪能力没有相较于 MLP 体现出显著优势，在中低噪声区间甚至可能不及后者。

3.4 Direction 5：Error Analysis and Visualization

本部分中，前十名代表性错分样本的选择规则为，取所有测试集错分样本中，模型对错误预测类别置信度最高的前十名。

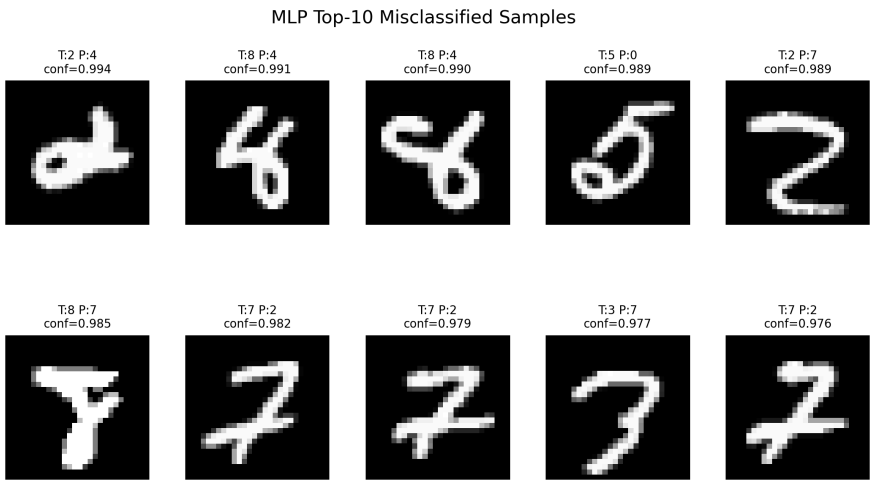
3.4.1 MLP baseline

下图为 MLP 基线模型在测试集上的混淆矩阵。



从 MLP 的混淆矩阵可以看到，主要错误集中在若干形状相近的类别对上：例如真实 `4` 被预测为 `9`、真实 `7` 被预测为 `2` 或 `9`、真实 `5` 被预测为 `3`、真实 `8` 被预测为 `3/6`。这说明 MLP 的主要错误模式不是随机错分，而是对局部笔画差异敏感、对整体结构相近类别区分不足，尤其在弯钩、闭环、竖笔画组合相似时更容易混淆。

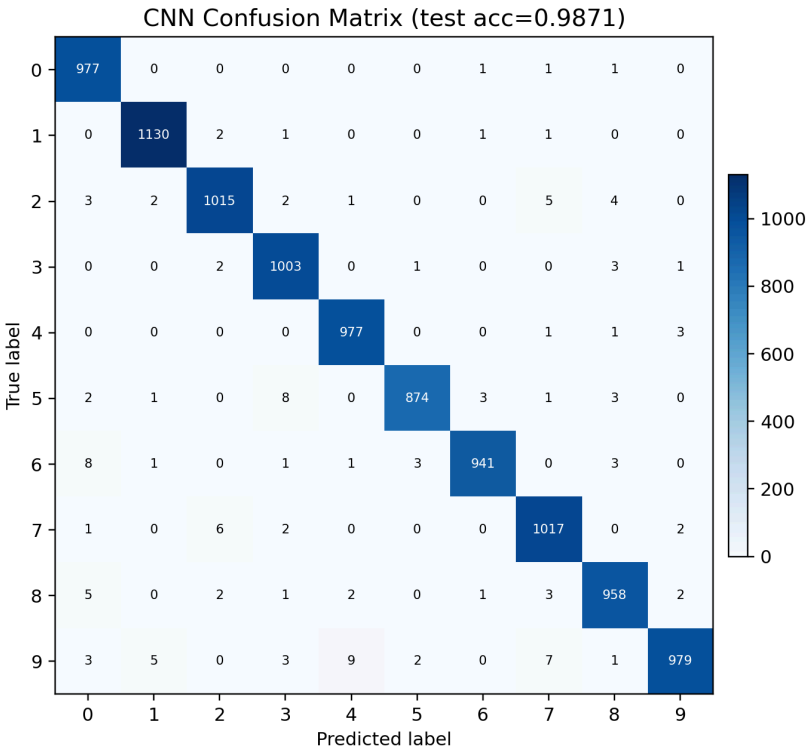
下图为 MLP 基线模型的 Top-10 错分样本。



Top-10 误分类样本中，高置信度错误大量出现在 `7→2` 、 `8→4` 、 `2→4/7` 、 `5→0` 等类别对，和混淆矩阵中的高频非对角项一致。可见 MLP 在部分样本上对局部特征感知能力不足，原因可能是输入展平后二维局部邻域关系被弱化，模型主要依赖全局像素组合做判别；当书写风格存在轻微形变、笔画粘连或缺损时，易把关键局部结构映射到错误类别模板，从而形成稳定的系统性错误。

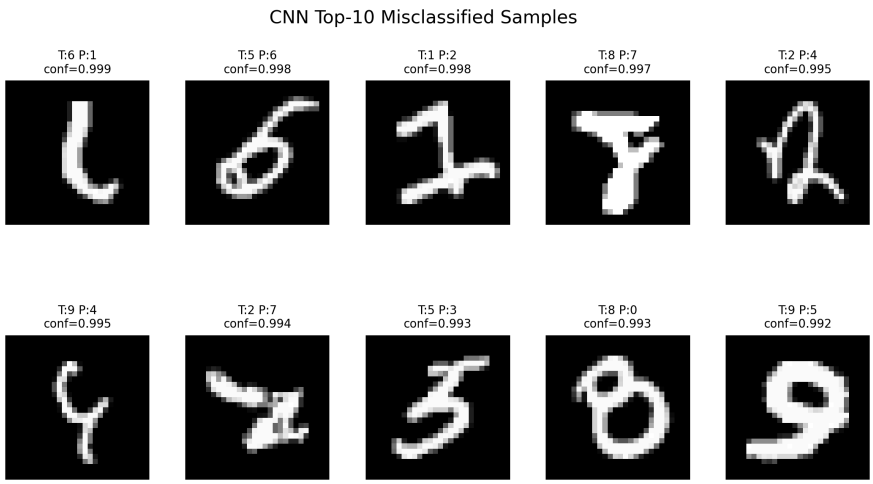
3.4.2 CNN baseline

下图为 CNN 基线模型在测试集上的混淆矩阵。



可以看到，CNN 的混淆矩阵整体更接近对角线，非对角项显著少于 MLP，说明总体分类更稳定。其主要残余错误仍集中在形状高度相近类别，如 `9→4/7` 、 `6→0` 、 `5→3` 、 `2→7` 、 `8→0` 。与 MLP 相比，CNN 在大多数类别对上的混淆明显减轻，尤其是对闭环结构和局部笔画组合的判别更准确。

下图为 CNN 基线模型的 Top-10 错分样本。



CNN 的 Top-10 错分样本多为极端书写体、笔画断裂/粘连明显或形状高度畸变的样本，且对应的混淆类别与混淆矩阵高频错分对一致，表明其错误更多是个体难区分样本所导致，而非普遍性区分失败。对比 MLP，CNN 显著避免了大量由局部结构误判引发的常见错分模式；但也会新增少量非主流书写风格下的高置信度错误。该差异与两者结构设计直接相关：CNN 通过局部连接、参数共享与池化对笔画局部模式更敏感、对小位移更鲁棒，因此整体错误率更低；而 MLP 的展平输入机制更容易丢失空间拓扑信息，导致在相似数字之间产生更系统的混淆。

4.附录

GitHub 代码链接：

- https://github.com/Jacky23307110248/CS30064-NeuralNetwork-DeepLearning/tree/main/PJ1

模型权重（ModelScope）：

- https://modelscope.cn/models/JackYHon/CS30064-PJ1/tree/master/best_models